Lista de exercícios

Lista dinâmica duplamente encadeada

Prof. Daniel Faustino Lacerda de Souza · danielfaustino@ci.ufpb.br

Instruções

- Utilize como base a biblioteca da pasta `04-lista-dinamica-duplamente-encadeada` do repositório da disciplina (github.com/Ensino-UFPB/exemplosed1).
- Cada função pedida deve ser escrita em `ListaDinEncadDupla.c`, com o protótipo acrescentado em `ListaDinEncadDupla.h`, e testada a partir do `main.c`.
- Toda função deve tratar a lista inválida (`li == NULL`), a lista vazia e a lista com um único elemento.

Questão 1 · Percurso do fim para o início

Implemente a função abaixo, que imprime as matrículas da lista do último para o primeiro elemento. A função deve chegar ao último elemento por `prox` e retornar ao início **exclusivamente** pelo campo `ant`.

```
void imprime_lista_inversa(Lista* li);
```

Para a lista 33 ⇄ 23 ⇄ 16, a saída é `16 23 33`. Para lista inválida ou vazia, a função imprime `(lista vazia)`. Indique o custo da função em função do número n de elementos.

Questão 2 · Verificação dos dois encadeamentos

Uma lista duplamente encadeada correta respeita duas propriedades: (i) o primeiro elemento tem `ant` igual a `NULL`; (ii) para todo elemento que possui sucessor, vale `no->prox->ant == no`. Implemente:

```
int lista_consistente(Lista* li);
```

A função devolve 1 se a lista respeita as duas propriedades e 0 caso contrário. Uma lista vazia é consistente; uma lista inválida (`li == NULL`) devolve 0. Utilize essa função para testar todas as questões seguintes.

Questão 3 · Identificação e correção de erros

A função a seguir foi escrita para remover o elemento de matrícula `mat`, mas contém erros.

```
int remove_lista(Lista* li, int mat){
    if(li == NULL) return 0;
    if((*li) == NULL) return 0;
    Elem* no = *li;
    while(no != NULL && no->dados.matricula != mat)
        no = no->prox;
    if(no == NULL) return 0;
    no->ant->prox = no->prox;
    free(no);
    return 1;
}
```

a) Descreva o que ocorre na chamada `remove_lista(li, 33)` sobre a lista 33 ⇄ 23 ⇄ 16.

b) Partindo novamente de 33 ⇄ 23 ⇄ 16, desenhe a lista após `remove_lista(li, 23)`, indicando os campos `ant` e `prox` de cada nó restante. Explique por que `tamanho_lista` devolve o valor esperado e por que `lista_consistente` (questão 2) devolve 0.

c) Reescreva a função corrigida.

Questão 4 · Remoção por posição

Implemente a função que remove o elemento da posição `pos`, em que a posição 1 corresponde ao primeiro elemento.

```
int remove_lista_pos(Lista* li, int pos);
```

A função devolve 1 em caso de sucesso e 0 se a lista for inválida ou vazia, ou se `pos` não corresponder a um elemento. O laço deve usar um único ponteiro de percurso: o antecessor deve ser obtido pelo próprio nó. Teste a remoção da primeira, de uma posição intermediária e da última posição, e indique o custo da função.

Questão 5 · Mover um elemento para o início

Caches do tipo LRU (*least recently used*) mantêm os itens em uma lista duplamente encadeada e, a cada acesso, movem o item acessado para o início. Implemente:

```
int move_para_inicio(Lista* li, int mat);
```

A função localiza o elemento de matrícula `mat` e o transfere para o início da lista **apenas religando ponteiros**, sem `malloc` e sem `free`. Devolve 1 se o elemento foi encontrado (inclusive quando ele já é o primeiro) e 0 caso contrário. Exemplo: 10 ⇄ 20 ⇄ 30 ⇄ 40 com `mat = 30` resulta em 30 ⇄ 10 ⇄ 20 ⇄ 40.

Quantos campos de ponteiro são escritos quando o elemento está no meio da lista? E quando é o último? Justifique pelo seu código.

Questão 6 · Remoção de todas as ocorrências

Implemente a função que remove da lista todos os elementos com matrícula `mat` e devolve a quantidade de elementos removidos.

```
int remove_todos(Lista* li, int mat);
```

Exemplo: 7 ⇄ 3 ⇄ 7 ⇄ 7 ⇄ 5 ⇄ 7 com `mat = 7` resulta em 3 ⇄ 5, e a função devolve 4. A lista deve ser percorrida uma única vez. Atenção: depois de `free(no)`, o campo `no->prox` não pode mais ser lido. Teste também uma lista em que todos os elementos têm a matrícula procurada.

Questão 7 · Inversão da lista no lugar

Implemente a função que inverte a ordem dos elementos da lista **sem alocar nem liberar nós** e sem copiar os dados entre nós.

```
void inverte_lista(Lista* li);
```

Exemplo: 1 ⇄ 2 ⇄ 3 ⇄ 4 ⇄ 5 resulta em 5 ⇄ 4 ⇄ 3 ⇄ 2 ⇄ 1, e a lista invertida deve passar em `lista_consistente`. Justifique por que, em uma lista duplamente encadeada, trocar os campos `ant` e `prox` de cada nó e atualizar o início é suficiente. Indique o custo da função.

Questão 8 · Sequência palíndroma

Implemente a função que devolve 1 se a sequência de matrículas da lista é a mesma quando lida do início para o fim e do fim para o início, e 0 caso contrário.

```
int eh_palindromo(Lista* li);
```

Use dois ponteiros, um partindo do primeiro e outro do último elemento, que caminham em sentidos opostos. Não é permitido usar vetor auxiliar nem alterar a lista. Exemplos: 1 ⇄ 2 ⇄ 3 ⇄ 2 ⇄ 1 e 4 ⇄ 8 ⇄ 8 ⇄ 4 devolvem 1; 1 ⇄ 2 ⇄ 2 ⇄ 3 devolve 0. A lista vazia e a lista de um elemento são palíndromas; a lista inválida devolve 0.

Defina com cuidado a condição de parada para listas de tamanho par e ímpar. Indique o custo da função e explique por que a mesma estratégia não se aplica diretamente à lista encadeada simples.

Questão extra · Desafio opcional (não pontuado)

O exercício 25 do capítulo 5 do livro-texto propõe uma **lista circular duplamente encadeada**: o `prox` do último aponta para o primeiro e o `ant` do primeiro aponta para o último. Mantendo `*li` como o endereço do primeiro elemento, o último passa a ser `(*li)->ant`.

Implemente `insere_lista_inicio`, `insere_lista_final`, `remove_lista_inicio` e `remove_lista_final` para essa representação, todas em O(1), e compare com os custos das mesmas operações na lista circular simples que guarda o início.